RISC-V Reliable Communication SoC: Test Plan

Oct 31, 2026 · @Vijay Yarabolu

Overview

The SoC passes verification when every message from the PC reaches the SPI slave unchanged, or the system reports FAIL after 3 retries. Corrupted data must never be reported as PASS.

Design under test (DUT): a VeeR EL2 RISC-V SoC that receives a payload over UART, checks it with CRC-16 (ARQ with ACK/NACK), buffers it in a FIFO, sends it over SPI, reads it back, re-checks the CRC and retries on mismatch.

In scope

- IP-level verification of CRC-16, FIFO, SPI (new), UART, GPIO and timer
- AXI4 interconnect and the AXI4 to AXI4-Lite bridge
- SoC-level C tests running on VeeR EL2
- Error injection on the UART input link and the SPI output link

Out of scope

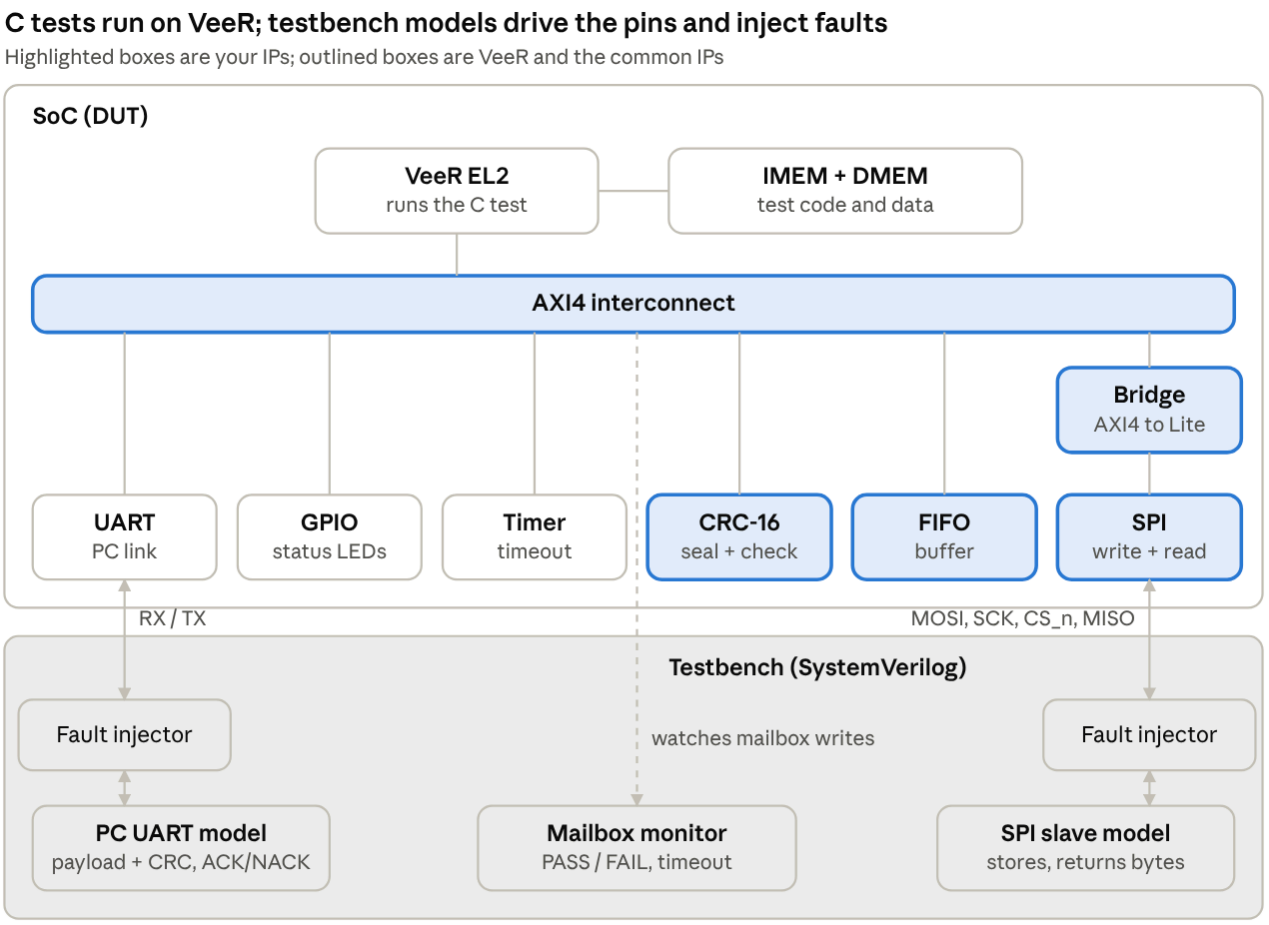
- Internal verification of the VeeR EL2 core (taken as proven, from CHIPS Alliance)
- FPGA timing closure and silicon tests
- Error correction in place (ECC); the project detects and retransmits only

Verification goals

1. Each IP meets its register map and protocol on its own.
2. The CPU reaches every IP at its address through the interconnect.
3. The 15-step communication flow passes end to end in simulation.
4. Every injected error is detected, and the retry or NACK path recovers it.

DUT and test environment

The whole SoC is the DUT; the testbench only touches its pins (UART and SPI) and watches one mailbox address on the bus.



SoC under test and testbench models

Faults are injected between the SoC pins and the models, so the same C test can run clean or with errors.

Interfaces under test

Interface	Between	Protocol
CPU bus	VeeR EL2 and interconnect	AXI4, 64-bit (to confirm from `soc_proj_DC/rtl`)
Peripheral bus	Bridge and SPI	AXI4-Lite, 32-bit
PC link	UART pins and PC model	8N1 serial, baud to confirm
SPI link	SPI pins and slave model	SPI mode 0, MSB first

The address map is still open and will be filled in from the interconnect RTL.

Verification strategy

Verification runs bottom up in three levels, and each level starts only when the one below it passes.

Level	What runs	Stimulus	Checker	Done when
L1 IP unit	Each IP alone in its own testbench	Bus BFM (AXI4-Lite or the interconnect's protocol) + pin drivers	Self-checking scoreboard, reference model	All IP test cases pass
L2 Integration	Interconnect + bridge + all IPs, no CPU	AXI4 master BFM in place of VeeR	Register read/write compare, address decode check	Every IP reachable at its address
L3 SoC C tests	Full SoC with VeeR EL2 running C firmware	C test program + testbench models (PC UART model, SPI slave model)	Test writes PASS/FAIL to a mailbox address; testbench ends the sim	All C tests pass, incl. fault injection

Reference models

- CRC-16-CCITT (poly 0x1021, init 0xFFFF) in the testbench and in Python for the PC script
- FIFO: a SystemVerilog queue
- SPI slave: a memory-backed model that stores written bytes and returns them on read
- PC: a UART model that sends payload + CRC and answers ACK/NACK

Pass/fail rule: a test passes only if the scoreboard sees zero mismatches, no X/Z on outputs, no bus error response (SLVERR/DECERR) unless the test expects one, and it finishes before its timeout.

Tools (to confirm): simulator (Verilator, Icarus, Questa or VCS) and the RISC-V GCC toolchain on the lab PC are still open; see Open items.

Features to verify

Each feature below maps to at least one test case in the next two sections.

IP	Features
CRC-16	CCITT polynomial 0x1021, init 0xFFFF; byte-wise input; result register; clear/restart; back-to-back packets
FIFO	Push/pop in order; full and empty flags; overflow and underflow handling; simultaneous push and pop; reset clears contents
SPI master	Byte transmit on MOSI; full-duplex receive on MISO; chip select low for the whole packet; SCK idle level and sample edge per mode; clock divider; TX ready / RX valid status; done interrupt
UART	RX of 8N1 frames at the chosen baud; TX of ACK (0x06), NACK (0x15) and report text; RX ready flag; framing and overrun flags (if the IP has them)
GPIO	Output write and readback; input read; direction control (if present)
Timer	Count up; compare match; interrupt; used for the retry timeout
Interconnect	Address decode to every slave; DECERR on unmapped address; read and write on all slaves; ID returned unchanged
AXI4 to Lite bridge	ID echo on BID/RID; RLAST = 1; 64 to 32-bit lane select by addr[2]; WSTRB shifted; SLVERR on burst (len > 0)
Firmware flow	CRC check #1 and ACK/NACK; original copy kept until PASS; SPI readback; CRC check #2; retry up to 3; PASS/FAIL report

IP-level test cases (L1 and L2)

There are 38 unit and integration tests; each one is self-checking and runs in its own testbench.

ID	IP	Test	Stimulus	External interface	Expected result
CRC-01	CRC-16	Reset values	Read result after reset	Reset	0xFFFF (init value)
CRC-02	CRC-16	Standard check vector	Feed "123456789"	None (internal)	0x29B1
CRC-03	CRC-16	Project vector	Feed 0x48 0x49 ("HI")	None (internal)	0x4087
CRC-04	CRC-16	Single-bit change	Feed 0x48 0x4A	None (internal)	0x70E4, differs from 0x4087
CRC-05	CRC-16	Residue check	Feed 0x48 0x49 0x40 0x87	None (internal)	0x0000
CRC-06	CRC-16	Clear between packets	Packet A, clear, packet B	None (internal)	B's CRC unaffected by A
CRC-07	CRC-16	Random packets	1,000 random packets, 1 to 64 bytes	None (internal)	Match reference model every time
FIFO-01	FIFO	Reset	Read flags after reset	Reset	empty = 1, full = 0
FIFO-02	FIFO	Order	Push 0x48 0x49 0x40 0x87, pop 4	None (internal)	Same bytes, same order
FIFO-03	FIFO	Full flag	Push DEPTH items	None (internal)	full = 1 after the last push
FIFO-04	FIFO	Overflow	Push when full	None (internal)	Data not overwritten; overflow flag if present
FIFO-05	FIFO	Underflow	Pop when empty	None (internal)	No pointer change; underflow flag if present
FIFO-06	FIFO	Simultaneous push/pop	Push and pop in the same cycle at half full	None (internal)	Count unchanged, order kept
FIFO-07	FIFO	Pointer wrap	Push/pop 3 × DEPTH items	None (internal)	No data loss across wrap
SPI-01	SPI	Reset state	Read pins after reset	SPI (`spi_cs_n`, `spi_sck`, `spi_mosi`), Reset	CS_n = 1, SCK = idle level, MOSI stable
SPI-02	SPI	Single byte TX	Write 0x49 to TX_DATA	SPI (`spi_mosi`, `spi_sck`, `spi_cs_n`)	Slave model receives 0x49, MSB first

ID	IP	Test	Stimulus	External interface	Expected result
SPI-03	SPI	Full duplex RX	Slave returns 0xA5 while master sends	SPI (`spi_miso` , `spi_mosi`)	RX_DATA = 0xA5
SPI-04	SPI	Packet with CS held	Send 4 bytes	SPI (`spi_cs_n` , `spi_sck`)	CS_n low for all 32 SCK edges, high after
SPI-05	SPI	Clock divider	Each divider setting	SPI (`spi_sck`), Clock	SCK period = clk / (2 × CLKS_PER_HALF_BIT)
SPI-06	SPI	Mode check	Mode 0 (and 1–3 if built)	SPI (`spi_sck` , `spi_mosi` , `spi_miso`)	Data sampled on the correct edge
SPI-07	SPI	Status and IRQ	One transfer	SPI (all four pins)	busy during, done IRQ once at the end
SPI-08	SPI	Back-to-back	16 bytes with no gap	SPI (all four pins)	All received in order
UART-01	UART	RX one frame	Model sends 0x48 at the set baud	UART (`uart_rx`)	RX_DATA = 0x48, ready = 1
UART-02	UART	TX one frame	Write 0x06 (ACK)	UART (`uart_tx`)	Model receives 0x06
UART-03	UART	Burst RX	4 bytes back to back	UART (`uart_rx`)	No byte lost
UART-04	UART	Baud mismatch	Model 3% off	UART (`uart_rx`)	Framing error flag, or the byte is caught later by CRC
GPIO-01	GPIO	Output	Write 0x0F	GPIO (`gpio_out`)	Pins = 0x0F, readback 0x0F
GPIO-02	GPIO	Input	Drive pins 0xA5	GPIO (`gpio_in`)	Read = 0xA5
TMR-01	Timer	Count	Enable, wait N cycles	Clock	Count = N ± 1
TMR-02	Timer	Compare IRQ	Set compare = 100	Clock	IRQ at cycle 100
IC-01	Interconnect	Decode	Write/read every slave base	None (internal)	Each slave sees only its own access
IC-02	Interconnect	Unmapped	Access a hole in the map	None (internal)	DECERR, no hang
IC-03	Interconnect	ID return	Accesses with IDs 0 to 15	None (internal)	BID/RID equal the request ID
IC-04	Interconnect	Back-to-back masters	IFU and LSU at once (if 2 masters)	None (internal)	Both complete, no deadlock

ID	IP	Test	Stimulus	External interface	Expected result
BR-01	Bridge	Lower word	Write addr[2] = 0	None (internal)	WDATA[31:0] reaches the slave
BR-02	Bridge	Upper word	Write addr[2] = 1	None (internal)	WDATA[63:32] reaches the slave, WSTRB[7:4] used
BR-03	Bridge	Read	Read any register	None (internal)	RLAST = 1, RID echoed, data on both halves
BR-04	Bridge	Burst rejected	AWLEN = 3	None (internal)	SLVERR, no hang

SoC-level C tests (L3)

There are 10 C programs; each one is compiled for VeeR EL2, loaded into instruction memory, and reports through a mailbox address the testbench watches.

ID	C test	What it does	Pass condition
C-01	`test_boot.c`	Reset, run from the reset vector, write PASS	Mailbox = PASS within 10,000 cycles
C-02	`test_gpio.c`	Write patterns to GPIO, read back	Pins and readback match
C-03	`test_timer.c`	Start timer, poll count, take the timer interrupt	Count grows; ISR runs once
C-04	`test_uart.c`	Send "OK" on TX; receive 4 bytes from the PC model	PC model sees "OK"; CPU reads the 4 bytes
C-05	`test_crc.c`	Run CRC-01 to CRC-05 vectors through the CRC registers	All 5 results match
C-06	`test_fifo.c`	Push 8, pop 8, check full/empty	Order and flags correct
C-07	`test_spi.c`	Send 4 bytes, read 4 back from the slave model	Readback = sent bytes
C-08	`test_main_flow.c`	Full 15-step flow with "HI": UART RX, CRC #1, ACK, FIFO, SPI write, readback, CRC #2, report	PC model gets ACK then "PASS, attempts 1"; slave holds 48 49 40 87

ID	C test	What it does	Pass condition
C-09	`test_arq_nack.c`	PC model corrupts the first UART send	CPU sends NACK once, ACK on the resend, then PASS
C-10	`test_spi_retry.c`	Testbench flips one MISO bit on the first readback	CRC #2 fails once, retry, then "PASS, attempts 2"

Mailbox convention (proposed): the C test writes 0x1 (PASS) or 0x2 (FAIL) to a reserved address; the testbench prints the result and stops the simulation. A second address can take characters for printf-style debug.

Error injection and negative tests

Every fault below must end in either a recovered PASS or a clean FAIL report, never a false PASS or a hang.

ID	Link	Fault injected (testbench)	Models	Expected result
ERR-01	UART in	Flip 1 bit in a payload byte	EMI spike	CRC #1 fails, NACK, PC resends, PASS
ERR-02	UART in	Flip 1 bit in the CRC bytes	EMI spike	NACK, resend, PASS
ERR-03	UART in	Drop one byte	Loose wire	CRC #1 fails or receive timeout, NACK
ERR-04	SPI out	Flip 1 MISO bit on readback	Noise on MISO	CRC #2 fails, retry, PASS on attempt 2
ERR-05	SPI out	Flip 1 MOSI bit on write	Noise on MOSI	Slave stores wrong byte, CRC #2 fails, retry, PASS
ERR-06	SPI out	Burst of 8 bits flipped	Crosstalk burst	Detected (CRC-16 catches all bursts up to 16 bits)
ERR-07	SPI out	One extra SCK pulse	Ringing	All bits shifted, CRC #2 fails, retry
ERR-08	SPI out	MISO stuck at 1	Open wire	Readback 0xFF, 3 retries fail, FAIL reported
ERR-09	SPI out	Slave corrupts stored byte	Bad flash cell	Readback differs, retry, PASS or FAIL after 3

ID	Link	Fault injected (testbench)	Models	Expected result
ERR-10	SPI out	Fault on every attempt	Permanent fault	Exactly 3 retries, then FAIL; no infinite loop
NEG-01	Bus	CPU reads an unmapped address	Firmware bug	DECERR, trap handled, no hang
NEG-02	FIFO	Firmware pushes past full	Firmware bug	Data protected, error flag
NEG-03	Timer	Retry timeout expires	Slave not responding	Retry counted, FAIL after limit

How faults are injected: a fault-injector module sits between the SoC pins and the PC or SPI slave model. A plusarg picks the fault, the byte and the bit (for example `+ERR=ERR-04 +BYTE=1 +BIT=0`), so the same C test runs clean or faulty.

Coverage plan

The sign-off target is 100% functional coverage of the points below and at least 90% line coverage on the project's own RTL (VeeR EL2 excluded).

Functional coverage points

Covergroup	Bins
CRC input	packet length 1, 2–8, 9–32, 33–64 bytes; byte values 0x00, 0xFF, other
FIFO level	empty, 1, half, DEPTH−1, full; push+pop same cycle
SPI transfer	1 byte, 4 bytes, 16 bytes; each built mode; each divider setting used
UART	ACK sent, NACK sent, report sent; RX bytes 0x00 and 0xFF
ARQ path	CRC #1 pass first try; fail then pass
SPI retry path	attempts = 1, 2, 3; final FAIL
Bus	read and write to every slave; DECERR; SLVERR from the bridge
Fault type	each ERR-01 to ERR-10 hit at least once

Code coverage: line, branch and FSM-state coverage on CRC, FIFO, SPI wrapper, bridge and interconnect. Any line left uncovered must be waived with a reason (for example, unreachable default case).

Test infrastructure

One `make` command builds a C test, loads it, runs the SoC simulation and prints PASS or FAIL; `make regress` runs every test.

SoC testbench parts

- `tb_soc_top.sv` : clock, reset, SoC instance, mailbox monitor, timeout
- `pc_uart_model.sv` : sends payload + CRC at the set baud, waits for ACK/NACK, resends on NACK
- `spi_slave_model.sv` : memory-backed slave that stores written bytes and returns them on read
- `fault_injector.sv` : sits on the UART and SPI pins, applies the fault chosen by plusargs
- `soc_assertions.sv` : CS_n stays low for a whole packet, no X on the bus, every AXI request gets a response

C test framework

- `crt0.S` : stack setup, jump to `main`
- `soc.h` : base addresses and register offsets for every IP
- `test_lib.c` : `pass()` , `fail(code)` , `print(str)` through the mailbox
- `link.ld` : places code in instruction memory and data in data memory

Build flow: C source → `riscv-gcc -march=rv32imc` → ELF → `objcopy` to hex → `$readmemh` into instruction memory → simulate.

Directory layout (proposed)

```
soc_proj_DC/
  rtl/            interconnect, bridge, soc_top
  IPs/            crc, fifo, spi, uart, gpio, timer
  RISC-V-VeeR-EL2/
  tb/
    unit/         one testbench per IP (L1)
    integ/        interconnect + IPs with AXI BFM (L2)
    soc/          tb_soc_top and models (L3)
  sw/
    common/       crt0.S, link.ld, soc.h, test_lib.c
    tests/        test_*.c
  scripts/        Makefile, regress list, pc_send.py
```

Regression: a list file names each test and its plusargs; the script runs them all and writes a pass/fail table plus the coverage report.

Entry, exit, risks and open items

Entry criteria (per level)

- L1: the IP's RTL compiles with no lint errors and its register map is written down
- L2: all L1 tests pass; address map frozen
- L3: L2 passes; C toolchain builds `test_boot.c`; VeeR boots from the reset vector

Exit criteria (sign-off)

- All 38 IP tests, 10 C tests and 13 error and negative tests pass in one regression run
- Functional coverage 100%, line coverage on project RTL at least 90%, waivers listed
- No open bug rated high

Milestones (in order; dates to be set with the guide)

1. IPs modified and unit tests passing (CRC, FIFO, SPI)
2. Interconnect + bridge integration passing (L2)
3. SoC boots and C-01 to C-07 pass
4. Main flow C-08 passes
5. Fault injection C-09, C-10 and ERR/NEG tests pass
6. Coverage closed, report written

Risks

Risk	Effect	Fallback
Interconnect protocol differs from AXI4	Bridge and SPI wrapper need rework	Read the interconnect RTL first, match the wrapper to it
Existing CRC uses a different polynomial	Vectors and PC script disagree	Fix CRC-16-CCITT (0x1021, 0xFFFF) across RTL, C and Python
No RISC-V toolchain on the lab PC	No L3 tests	Install the prebuilt riscv-gnu-toolchain, or build hex files elsewhere
VeeR EL2 simulation is slow	Long regressions	Keep L3 tests short; do bulk random tests at L1
Simulator lacks SystemVerilog assertions or coverage	No coverage numbers	Use Verilator with coverage, or Questa if available

Open items

- [ ] Confirm the interconnect's bus protocol and address map from `soc_proj_DC/rtl`
- [ ] Confirm the simulator on the lab PC
- [ ] Confirm the RISC-V GCC toolchain (`which riscv64-unknown-elf-gcc`)
- [ ] Confirm the CRC IP's polynomial and interface
- [ ] Confirm the FIFO depth and width
- [ ] Pick the mailbox address for PASS/FAIL